

# DATA PREPROCESSING

*A Simple Student Guide for Cleaning CSV / Tabular Data with Pandas & NumPy*

## 1. What is Preprocessing?

Preprocessing means cleaning and organizing raw data BEFORE you use it for analysis or machine learning. Raw datasets are often messy — missing values, wrong data types, duplicate rows, or unreadable column names. If you feed messy data into a model, you get wrong or misleading results.

**Simple Rule: "Garbage In = Garbage Out." Clean data first, analyze second.**

### Basic Steps of Preprocessing

| Step                     | What It Means                                                |
|--------------------------|--------------------------------------------------------------|
| 1. Understand the data   | Check shape, column names, data types (df.info(), df.head()) |
| 2. Handle missing values | Fill or remove empty (NaN) cells                             |
| 3. Remove duplicates     | Delete repeated rows                                         |
| 4. Fix data types        | Convert text to number/date where needed                     |
| 5. Handle outliers       | Detect and treat extreme, unusual values                     |
| 6. Encode categories     | Convert text categories into numbers for ML                  |
| 7. Scale numbers         | Bring numeric columns to the same range                      |
| 8. Visualize             | Plot graphs to understand patterns and errors                |

## 2. Graphs for Numerical vs Categorical Data

Before and after cleaning, always LOOK at your data using graphs. It helps you spot missing values, outliers, and wrong entries visually.

### A. Numerical Data (e.g., Age, Price, Marks)

| Graph        | Purpose                                    | Code (matplotlib/seaborn)                                      |
|--------------|--------------------------------------------|----------------------------------------------------------------|
| Histogram    | See how values are distributed (spread)    | <pre>df['age'].hist()<br/>plt.show()</pre>                     |
| Box Plot     | Spot outliers and see min/max/median       | <pre>sns.boxplot(x=df['price'])<br/>plt.show()</pre>           |
| Scatter Plot | See relationship between 2 numeric columns | <pre>plt.scatter(df['age'], df['salary'])<br/>plt.show()</pre> |
| Line Plot    | See trend over time/order                  | <pre>df['sales'].plot(kind='line')<br/>plt.show()</pre>        |

## B. Categorical Data (e.g., Gender, City, Product Type)

| Graph      | Purpose                                  | Code (matplotlib/seaborn)                                                            |
|------------|------------------------------------------|--------------------------------------------------------------------------------------|
| Bar Chart  | Compare count/frequency of each category | <code>df['city'].value_counts().plot(kind='bar')</code><br><code>plt.show()</code>   |
| Pie Chart  | Show proportion (%) of each category     | <code>df['gender'].value_counts().plot(kind='pie')</code><br><code>plt.show()</code> |
| Count Plot | Same as bar chart, seaborn style         | <code>sns.countplot(x='city', data=df)</code><br><code>plt.show()</code>             |

## 3. How to Fill / Handle Missing Values

Missing values appear as NaN (Not a Number) in pandas. First, always check how many are missing:

```
df.isnull().sum()      # count of missing values per column
df.isnull().mean() * 100 # percentage missing per column
```

### Methods to Handle Missing Values

| Method           | When to Use                            | Code                                                                |
|------------------|----------------------------------------|---------------------------------------------------------------------|
| Remove rows      | Very few rows have missing data (< 5%) | <code>df.dropna(inplace=True)</code>                                |
| Remove column    | A whole column is mostly empty (> 50%) | <code>df.drop('column_name', axis=1, inplace=True)</code>           |
| Fill with Mean   | Numeric column, no extreme outliers    | <code>df['age'].fillna(df['age'].mean(), inplace=True)</code>       |
| Fill with Median | Numeric column WITH outliers           | <code>df['price'].fillna(df['price'].median(), inplace=True)</code> |
| Fill with Mode   | Categorical (text) column              | <code>df['city'].fillna(df['city'].mode()[0], inplace=True)</code>  |
| Forward Fill     | Time-series / ordered data             | <code>df.fillna(method='ffill', inplace=True)</code>                |
| Interpolate      | Numeric, smoothly changing data        | <code>df['temp'].interpolate(inplace=True)</code>                   |

### Worked Example

**Problem:** A student dataset has missing values in 'Marks' (numeric) and 'City' (categorical).

**Sample data:**

| Name | Marks | City   |
|------|-------|--------|
| Amit | 78    | Delhi  |
| Riya | NaN   | Mumbai |

| Name  | Marks | City  |
|-------|-------|-------|
| Sohan | 65    | NaN   |
| Priya | 82    | Delhi |

#### Solution:

```
import pandas as pd

df = pd.read_csv('students.csv')

# Fill numeric column with median
df['Marks'].fillna(df['Marks'].median(), inplace=True)

# Fill categorical column with mode (most frequent value)
df['City'].fillna(df['City'].mode()[0], inplace=True)

# Confirm no missing values remain
print(df.isnull().sum())
```

## 4. Other Things Students Must Know

---

### A. Remove Duplicate Rows

**Problem:** Same student record appears twice in the CSV file.

```
print(df.duplicated().sum()) # check how many duplicates
df.drop_duplicates(inplace=True)
```

### B. Fix Wrong Data Types

**Problem:** 'Date' column or 'Price' column is stored as text (object) instead of proper type.

```
df.dtypes # check current types
df['Date'] = pd.to_datetime(df['Date']) # text -> date
df['Price'] = df['Price'].astype(float) # text -> number
```

### C. Detect and Handle Outliers (IQR Method)

**Problem:** One student's 'Marks' shows 500 out of 100 — an impossible outlier.

```
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
```

```
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df = df[(df['Marks'] >= lower) & (df['Marks'] <= upper)]
```

## D. Encode Categorical Columns (for Machine Learning)

**Problem:** Model needs numbers, but 'Gender' column has text values 'Male'/'Female'.

```
# One-Hot Encoding (creates new 0/1 columns)
df = pd.get_dummies(df, columns=['Gender'])

# OR Label Encoding (converts to 0,1,2...)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
```

## E. Scale Numeric Columns

**Problem:** 'Salary' is in thousands while 'Age' is in tens — models get confused by different scales.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])
```

## F. Clean Column Names

**Problem:** Column names have spaces/capitals like 'Student Name', 'MARKS (%)' — hard to code with.

```
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')
# 'Student Name' -> 'student_name'
```

## G. Always Explore First

| Command       | What It Tells You                         |
|---------------|-------------------------------------------|
| df.head()     | First 5 rows — quick look                 |
| df.shape      | Number of rows and columns                |
| df.info()     | Column names, data types, non-null counts |
| df.describe() | Mean, min, max, std for numeric columns   |

| Command                            | What It Tells You             |
|------------------------------------|-------------------------------|
| <code>df.isnull().sum()</code>     | Missing values per column     |
| <code>df.duplicated().sum()</code> | Number of duplicate rows      |
| <code>df['col'].unique()</code>    | Unique categories in a column |

## 5. Minimum Checklist Before Using a Dataset

---

- Loaded the CSV and checked `df.head()`, `df.info()`, `df.shape`
- Checked and handled all missing values (`fillna` / `dropna`)
- Removed duplicate rows
- Fixed incorrect data types (numbers, dates)
- Checked and treated outliers in important numeric columns
- Visualized numeric columns (histogram / box plot)
- Visualized categorical columns (bar / pie / count plot)
- Encoded categorical columns if using Machine Learning
- Scaled numeric columns if using Machine Learning
- Cleaned messy column names

*Tip for Students: Never skip visualization. A single graph often reveals a data problem that would take many lines of code to find otherwise.*